TEC | Tecnológico de Costa Rica

Proyecto 2

Estructuras de Datos

Santiago Eloy Céspedes Mora
José Pablo Barrantes Hernández
Jorge Ulate Mora

24/06/2026

1. Descripción del problema

Este proyecto se centra en la creación y gestión de una red galáctica, un sistema caracterizado por su inmensidad y la complejidad de sus diversas rutas interconectadas. La galaxia se modela como una estructura dinámica compuesta por nodos tales como estrellas, planetas, nebulosas y aristas que representan las rutas espaciales entre ellos. Esta configuración presenta un desafío significativo para la navegación eficiente. Por consiguiente, es necesario desarrollar un sistema capaz de procesar esta red para calcular las rutas óptimas, gestionar viajes intergalácticos, almacenar datos de forma persistente y visualizar conexiones mediante la implementación de una variante del algoritmo de Kruskal. Asimismo, el sistema debe gestionar los datos obtenidos a través de una API externa, garantizando la integridad de la información y evitando redundancias en el registro de naves y rutas.

2. Justificación

La optimización del tiempo de desplazamiento en entornos complejos es una necesidad crítica para los Guardianes de la Galaxia, quienes requieren una solución robusta para coordinar el transporte entre sistemas estelares. La implementación de algoritmos de rutas mínimas y árboles de expansión es fundamental para minimizar el costo de los viajes en una red que posee billones de posibles conexiones. Para organizar la información, es necesario utilizar estructuras de datos dinámicas, como listas o matrices, permitiendo realizar operaciones CRUD de manera eficiente. Adicionalmente, el desarrollo de este proyecto fomenta el análisis crítico, ya que exige la validación de rutas y la aplicación técnica de algoritmos complejos, tales como el algoritmo de Kruskal modificado de forma aleatoria.

3. Solución propuesta

3.1. Ambiente de desarrollo

El desarrollo del sistema se llevó a cabo utilizando las siguientes herramientas de software, seleccionadas por su robustez y eficiencia:

- **C++:** Lenguaje de programación de propósito general, orientado a objetos, que destaca por su alto rendimiento y control preciso sobre la gestión de memoria. Su capacidad para manejar punteros dinámicos resulta esencial para implementar estructuras de datos complejas como grafos, garantizando la optimización de recursos.
- **Visual Studio:** Entorno de Desarrollo Integrado (IDE) de Microsoft, utilizado para la compilación y depuración del sistema. Este entorno facilita la gestión de librerías

externas necesarias para el consumo de servicios web mediante *curl*, proporcionando herramientas avanzadas para asegurar la estabilidad del código.

3.2. Estructuras de datos utilizadas

Para la modelación lógica de la red galáctica y las entidades asociadas, se definieron estructuras (*struct*) personalizadas que permiten la persistencia en memoria y la vinculación eficiente entre nodos y registros.

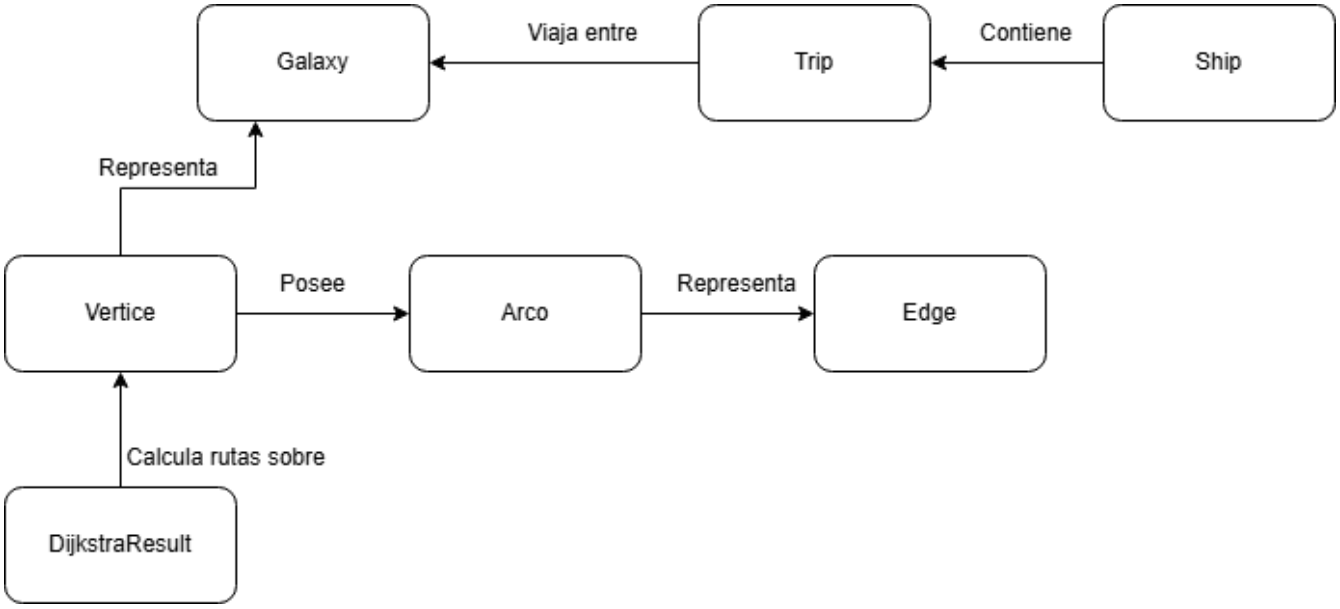
Estructura	**Descripción**
`Vertice`	Representa un nodo galaxia; contiene el identificador, punteros a la sub-lista de arcos y al siguiente nodo del grafo.
`Arco`	Define las conexiones (aristas) entre galaxias, incluyendo el costo del trayecto.
`Ship`	Entidad que agrupa la información de las naves, incluyendo su historial de viajes y estado operativo.
`Trip`	Almacena el registro detallado de un viaje específico realizado por una nave.
`Galaxy`	Estructura para almacenar atributos espaciales (coordenadas X, Y, Z) y datos de identificación.
`Edge`	Definición técnica de las rutas (aristas), incluyendo tiempo de duración y estado de la conexión.
`DijkstraResult`	Estructura auxiliar diseñada para el cálculo y almacenamiento de rutas óptimas entre galaxias.

3.3. Funciones principales

El sistema se organiza en funciones modulares encargadas de la gestión de entidades y la lógica algorítmica:

- **Gestión CRUD:** Funciones dedicadas a la creación, lectura, actualización y eliminación de galaxias, naves y rutas.
- **Algoritmo de Kruskal:** Función especializada que implementa la variante aleatoria para la generación de la red galáctica de bajo costo.
- **Consumo de API:** Funciones que utilizan la librería *curl* para extraer datos en formato JSON desde el servicio *Mock API*.
- **Reportes:** Funciones encargadas de la escritura de archivos de texto con los resultados de las rutas optimizadas y estadísticas de conectividad.

3.4. Diagrama de estructuras



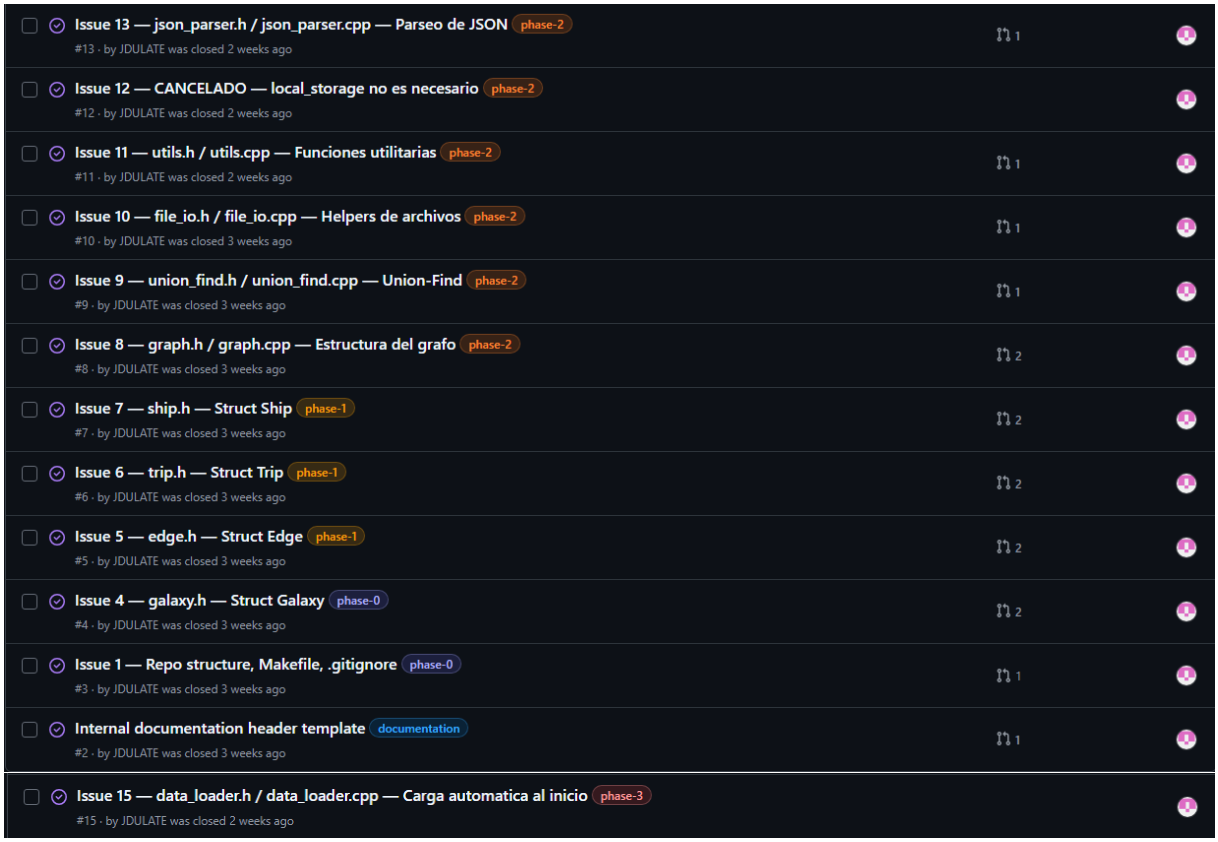
3.5. Minuta de trabajo Santiago



3.6. Minuta de trabajo José Pablo



3.7. Minuta de trabajo Jorge



4. Análisis de resultados

A continuación, se presenta un análisis del cumplimiento de los requerimientos planteados en el proyecto, evaluando las funcionalidades implementadas y su estado final.

Requerimiento	Estado	Observaciones
Estructuras de Datos		
Grafo mediante lista de adyacencia (Vértices y Arcos)	Completo	Representación dinámica del mapa galáctico mediante listas enlazadas.
Galaxias	Completo	Almacena identificador, nombre y coordenadas espaciales.
Rutas (Edge)	Completo	Contiene información del costo, duración y estado de las conexiones.

Requerimiento	Estado	Observaciones
X-Wings (Ship)	Completo	Registro de naves con historial de viajes asociado.
Viajes (Trip)	Completo	Almacena los recorridos realizados entre galaxias.
Resultado de Dijkstra	Completo	Estructura auxiliar para almacenar caminos mínimos calculados.
Operaciones CRUD		
CRUD de galaxias	Completo	Permite agregar, modificar, eliminar y consultar galaxias.
CRUD de rutas	Completo	Administración completa de conexiones entre galaxias.
CRUD de naves	Completo	Gestión de la información de las naves registradas.
CRUD de viajes	Completo	Registro y mantenimiento del historial de viajes.
Validación de datos duplicados	Completo	Se evita el registro de identificadores repetidos.
Algoritmos		
Algoritmo de Dijkstra	Completo	Determina la ruta de menor costo entre dos galaxias.
Algoritmo de Kruskal Modificado	Completo	Construcción del árbol de expansión utilizando aristas aleatorias proporcionadas por la API.
Recorrido DFS	Completo	Obtiene todas las rutas posibles desde una galaxia origen.
Union-Find	Completo	Utilizado para detectar ciclos durante la ejecución de Kruskal.

Requerimiento	Estado	Observaciones
Consultas		
Mostrar todas las rutas posibles	Completo	Implementado mediante recorrido en profundidad (DFS).
Ruta de menor costo	Completo	Consulta basada en el algoritmo de Dijkstra.
Árbol de expansión mínima	Completo	Presenta el resultado obtenido por Kruskal.
Historial de viajes por nave	Completo	Consulta de todos los viajes registrados para una nave.
Reportes		
Reporte de rutas más cortas	Completo	Generación automática del archivo de resultados de Dijkstra.
Reporte del árbol optimizado	Completo	Exportación del árbol de expansión mínima generado por Kruskal.
Reporte de estadísticas	Completo	Incluye información sobre rutas calculadas y conectividad del grafo.
Almacenamiento en archivos txt	Completo	Los resultados son persistidos en archivos de salida.
Carga y Persistencia de Datos		
Consumo de API REST	Completo	Obtención automática de galaxias y rutas mediante servicios web.
Procesamiento de JSON	Completo	Interpretación y carga de la información recibida desde la API.
Carga automática del grafo	Completo	Construcción dinámica del grafo a partir de los datos obtenidos.

Requerimiento	Estado	Observaciones
Lectura y escritura de archivos	Completo	Manejo de archivos de configuración y reportes del sistema.
Resultados Generales		
Representación correcta del grafo	Completo	Implementación mediante estructuras dinámicas y listas de adyacencia.
Organización modular del sistema	Completo	Código dividido por módulos, entidades, algoritmos y utilidades.
Uso de estructuras dinámicas	Completo	Implementación basada en punteros sin utilizar contenedores STL para la estructura principal.
Cumplimiento de los objetivos del proyecto	Completo	El sistema satisface los requerimientos establecidos en el enunciado del proyecto.

5. Conclusiones

- **Gestión de Memoria y Estructuras Dinámicas:** La implementación basada exclusivamente en el uso de punteros y estructuras personalizadas permitió un control eficiente de los recursos del sistema. También permitiendo comprender como funcionan este tipo de estructuras en profundidad y su gestión en C++.

- **Eficacia Algorítmica:** Se logró integrar con éxito algoritmos de optimización complejos, tales como el Kruskal modificado y Dijkstra. La correcta implementación de estructuras auxiliares, como la técnica Union-Find, fue determinante para garantizar la integridad de los resultados y la eficiencia en la búsqueda de rutas mínimas en redes de gran escala, demostrando la aplicabilidad de los grafos en problemas de conectividad real.

- **Análisis Crítico y Resolución de Problemas:** El proyecto permitió desarrollar un pensamiento analítico frente a la validación de integridad de datos. La implementación de filtros para evitar registros duplicados, el manejo de excepciones y la

persistencia de datos en archivos txt aseguran que el programa mantenga un estado consistente, cumpliendo satisfactoriamente con todos los objetivos que se solicitaron.

6. Recomendaciones

- **Persistencia en Base de Datos:** Se podría implementar una base de datos para realizar consultas y manejar volúmenes de datos de forma más eficiente.
- **Optimización de Algoritmos:** Para escenarios de galaxias con una densidad de conexiones extremadamente alta, se podrían usar otras estructuras más avanzadas para mejorar la complejidad de los algortimos.

7. Bibliografía

Referencias

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*, 269–271. https://doi.org/10.1007/BF01386390

[2] ISO/IEC. (2020). *Programming languages — C++* (Standard No. ISO/IEC 14882:2020). https://isocpp.org/std/the-standard

[3] Stenberg, D. (2026). *libcurl - the multiprotocol file transfer library.* https://curl.se/libcurl/

[4] Tecnológico de Costa Rica. (2026). *Proyecto programado: Grafos y Archivos.* Centro Académico de Alajuela. https://galaxias-mock-api.onrender.com/